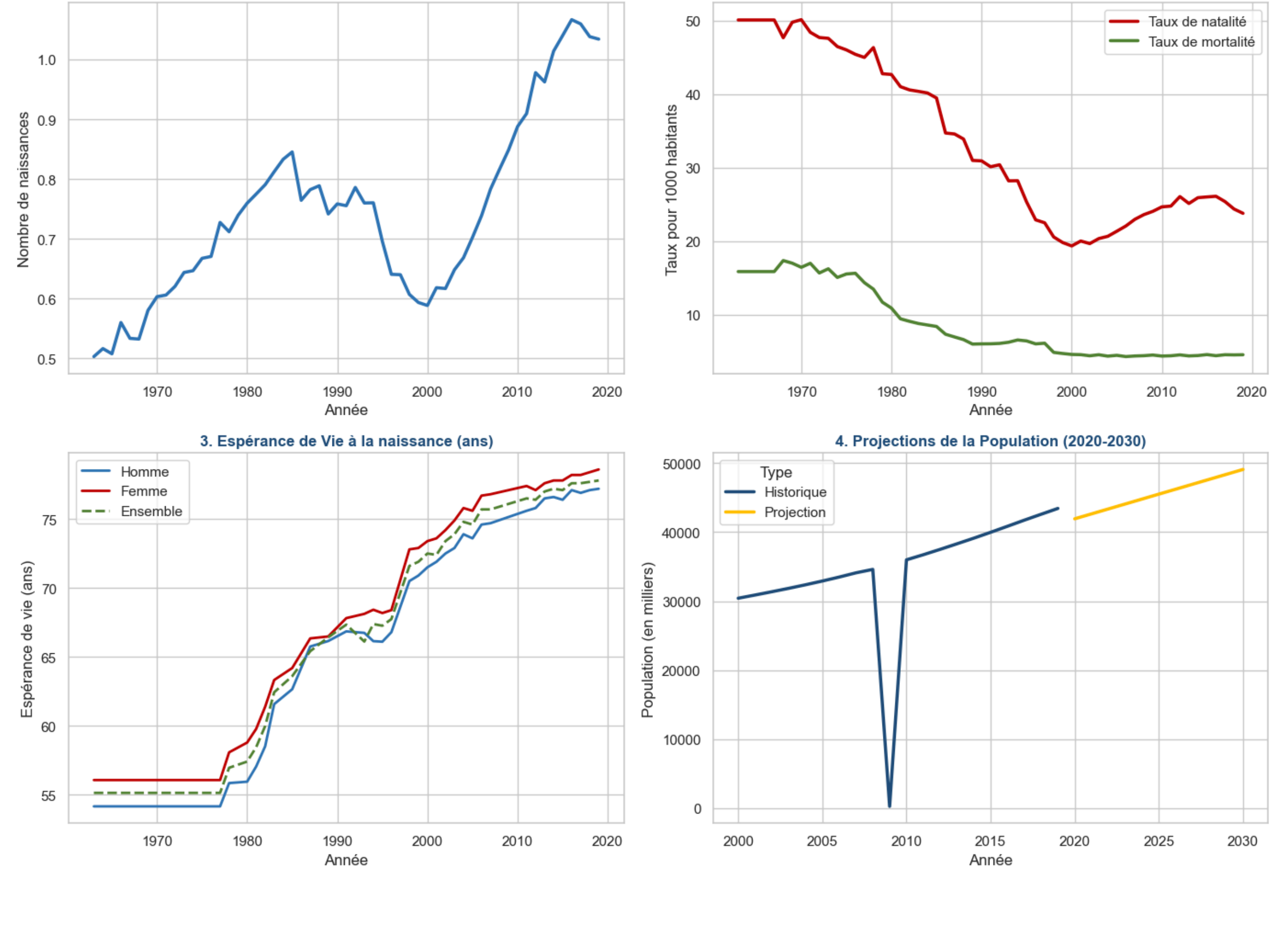
In [5]:

```python
# -*- coding: utf-8 -*-
"""
=== CODE UNIQUE POUR L'ANALYSE DÉMOGRAPHIQUE DE L'ALGÉRIE ===
Ce script est conçu pour être exécuté soit en tant que fichier Python standard (.py),
soit en copiant ses sections dans un notebook Jupyter (.ipynb).

Il réalise les tâches suivantes :
1. Extraction et nettoyage des données à partir de 'extracted_text.txt'
2. Fusion et interpolation linéaire des valeurs manquantes
3. Analyse statistique descriptive complète (Moyenne, Médiane, Écart-type, TCAM, etc.)
4. Modélisation et prévisions démographiques pour la période 2020-2030
5. Génération d'un fichier Excel (.xlsx) hautement stylisé avec 10 feuilles et graphiques ONS intégrés
6. (Optionnel pour Jupyter) Visualisation directe des graphiques clés dans la console/notebook.
"""
!pip install xlsxwriter
import os
import re
import math
import numpy as np
import pandas as pd
from scipy import stats
import xlsxwriter

# --- CONFIGURATION DES CHEMINS (Avec détection automatique) ---
# Détecte automatiquement si le fichier est dans le même dossier
dossier_courant = os.path.dirname(os.path.abspath(__file__)) if '__file__' in locals() else os.getcwd()

chemin_txt = os.path.join(dossier_courant, 'extracted_text.txt')
if not os.path.exists(chemin_txt):
    # Chemin de secours absolu si exécuté depuis un autre dossier sous Windows
    chemin_txt = r'c:\Users\hp\Downloads\Hakim333\extracted_text.txt'

chemin_excel = os.path.join(dossier_courant, 'Memoire_Demographie_Algerie_v3.xlsx')

print(f"[-] Lecture du fichier texte : {chemin_txt}")
if not os.path.exists(chemin_txt):
    raise FileNotFoundError(f"Le fichier 'extracted_text.txt' est introuvable. Veuillez vérifier son emplacement.")

with open(chemin_txt, 'r', encoding='utf-8') as f:
    text = f.read()

# --- FONCTION UTILITAIRE DE NETTOYAGE ---
def cn(s):
    """Nettoie une chaîne de caractères et la convertit en float."""
    s = s.strip().replace('\u202f','').replace(' ','').replace(',','.').replace('*','').replace('\xa0','')
    try:
        return float(s)
    except:
        return np.nan

# ==============================================================================
# SECTION 1 : EXTRACTION DES DONNÉES DE L'ONS depuis le texte brut
# ==============================================================================
print("[1/5] Extraction des données démographiques...")

# --- 1.1 Mouvement Naturel (Tableau 1) ---
t1 = []
idx_t1 = text.find('Tableau 1: Mouvement naturel')
idx_t2 = text.find('Tableau 2:')
block1 = text[idx_t1:idx_t2].split('\n')
i = 0
while i < len(block1):
    ln = block1[i].strip().replace('*','').strip()
    if re.match(r'^\d{4}$', ln) and 1963 <= int(ln) <= 2019:
        y = int(ln)
        t1.append({
            'Année': y,
            'Naissances': cn(block1[i+1]),
            'Décès': cn(block1[i+2]),
            'Mort_nés': cn(block1[i+3]),
            'Mariages': cn(block1[i+4])
        })
        i += 4
    i += 1
df1 = pd.DataFrame(t1)

# --- 1.2 Indicateurs Démographiques (Tableau 2) ---
# Correction manuelle des données 1967-1970 à cause d'un décalage ONS dans le texte brut
t2 = []
manual_t2 = {
    1967: {'TN': 50.12, 'TM': 15.87, 'AN': 34.2},
    1968: {'TN': 47.70, 'TM': 17.37, 'AN': 30.3},
    1969: {'TN': 49.81, 'TM': 17.01, 'AN': 32.8},
    1970: {'TN': 50.16, 'TM': 16.45, 'AN': 33.7},
}
for yr, vals in manual_t2.items():
    t2.append({
        'Année': yr,
        'Naiss_corr': np.nan,
        'Taux_natalité': vals['TN'],
        'Décès_corr': np.nan,
        'Taux_mortalité': vals['TM'],
        'Accroissement_naturel': vals['AN']
    })

idx_t3 = text.find('Tableau 3:')
block2 = text[text.find('Tableau 2: Indicateurs'):idx_t3].split('\n')
i = 0
while i < len(block2):
    ln = block2[i].strip().replace('*','').strip()
    if re.match(r'^\d{4}$', ln) and 1971 <= int(ln) <= 2019:
        y = int(ln)
        vals = []
        for k in range(1, 7):
            if i+k < len(block2):
                vals.append(cn(block2[i+k]))
        if len(vals) >= 5:
            t2.append({
                'Année': y,
                'Naiss_corr': vals[0],
                'Taux_natalité': vals[1],
                'Décès_corr': vals[2],
                'Taux_mortalité': vals[3],
                'Accroissement_naturel': vals[4]
            })
        i += 5
    i += 1
df2 = pd.DataFrame(t2).sort_values('Année').reset_index(drop=True)

# --- 1.3 Évolution de la population (Tableau 40) ---
t40 = []
block40 = text[text.find('Tableau 40: Evolution de la population'):text.find('Tableau  41:')].split('\n')
i = 0
while i < len(block40):
    ln = block40[i].strip().replace('*','')
    if re.match(r'^\d{4}$', ln) and 1966 <= int(ln) <= 2019:
        t40.append({
            'Année': int(ln),
            'Population_milliers': cn(block40[i+1])
        })
        i += 1
    i += 1
df40 = pd.DataFrame(t40)

# --- 1.4 Population par sexe (Tableau 41) ---
t41 = []
block41 = text[text.find('Tableau  41: R'):text.find('Tableau 42:')].split('\n')
i = 0
while i < len(block41):
    ln = block41[i].strip().replace('*','')
    if re.match(r'^\d{4}$', ln) and 1966 <= int(ln) <= 2019:
        y = int(ln)
        vals = []
        for k in range(1, 5):
            if i+k < len(block41):
                v = cn(block41[i+k])
                if not math.isnan(v):
                    vals.append(v)
        if len(vals) >= 2:
            masc = vals[0]
            fem = vals[1]
            t41.append({
                'Année': y,
                'Pop_Masculin': masc,
                'Pop_Féminin': fem,
                'Pop_Total': masc + fem
            })
        i += 3
    i += 1
df41 = pd.DataFrame(t41)

# --- 1.5 Espérance de vie & quotients de mortalité (Tables de mortalité) ---
esp = []
matches = list(re.finditer(r'Tableau \d+: Table de mortalit.*?Ann.e (\d{4})', text))
for idx, m in enumerate(matches):
    year = int(m.group(1))
    s = m.start()
    e = matches[idx+1].start() if idx+1 < len(matches) else s + 10000
    sub = text[s:e].split('\n')
    rec = {'Année': year, 'Ex_Ens': np.nan, 'Ex_H': np.nan, 'Ex_F': np.nan, 'Qx_Ens': np.nan, 'Qx_H': np.nan, 'Qx_F': np.nan}
    j = 0
    while j < len(sub):
        section = None
        if 'Ensemble / Total' in sub[j] or ('Ensemble' in sub[j] and 'Male' not in sub[j] and 'Female' not in sub[j]):
            section = 'ens'
        elif 'Masculin' in sub[j]:
            section = 'h'
        elif 'minin' in sub[j] or 'Female' in sub[j]:
            section = 'f'

        if section:
            while j < len(sub) and '0 an' not in sub[j]:
                j += 1
            if j < len(sub):
                vals = []
                for k in range(1, 8):
                    if j+k < len(sub):
                        v = cn(sub[j+k])
                        if not math.isnan(v):
                            vals.append(v)
                if len(vals) >= 6:
                    if section == 'ens':
                        rec['Qx_Ens'] = vals[0]
                        rec['Ex_Ens'] = vals[-1]
                    elif section == 'h':
                        rec['Qx_H'] = vals[0]
                        rec['Ex_H'] = vals[-1]
                    elif section == 'f':
                        rec['Qx_F'] = vals[0]
                        rec['Ex_F'] = vals[-1]
        j += 1
    esp.append(rec)
df_esp = pd.DataFrame(esp)

# ==============================================================================
# SECTION 2 : FUSION ET INTERPOLATION DES VALEURS MANQUANTES
# ==============================================================================
print("[2/5] Structuration, fusion et nettoyage de la base de données...")

years_df = pd.DataFrame({'Année': range(1963, 2020)})
df_merged = years_df.merge(df1, on='Année', how='left') \
                    .merge(df2[['Année', 'Taux_natalité', 'Taux_mortalité', 'Accroissement_naturel']], on='Année', how='left') \
                    .merge(df40, on='Année', how='left') \
                    .merge(df41, on='Année', how='left') \
                    .merge(df_esp, on='Année', how='left')

# Version nettoyée (Interpolation linéaire des valeurs vides)
df_clean = df_merged.copy()
for col in df_clean.columns:
    if col != 'Année':
        df_clean[col] = pd.to_numeric(df_clean[col], errors='coerce')
df_clean = df_clean.interpolate(method='linear', limit_direction='both')

# ==============================================================================
# SECTION 3 : ANALYSE STATISTIQUE DESCRIPTIVE
# ==============================================================================
print("[3/5] Calcul de l'analyse descriptive et du TCAM...")

indicators = {
    'Naissances': 'Naissances',
    'Décès': 'Décès',
    'Taux_natalité': 'Taux de natalité (‰)',
    'Taux_mortalité': 'Taux de mortalité (‰)',
    'Accroissement_naturel': 'Accroissement naturel (‰)',
    'Ex_Ens': 'Espérance de vie (ans)',
    'Population_milliers': 'Population (milliers)'
}

stat_rows = []
for col, label in indicators.items():
    s = df_clean[col].dropna()
    if len(s) < 2:
        continue
    first_v = s.iloc[0]
    last_v = s.iloc[-1]
    n_years = len(s)

    evol = (last_v - first_v) / first_v * 100
    tcam = ((last_v / first_v) ** (1 / n_years) - 1) * 100 if first_v > 0 else np.nan

    yr_min = df_clean.loc[df_clean[col].idxmin(), 'Année']
    yr_max = df_clean.loc[df_clean[col].idxmax(), 'Année']

    stat_rows.append({
        'Indicateur': label,
        'Moyenne': round(s.mean(), 2),
        'Médiane': round(s.median(), 2),
        'Écart-type': round(s.std(), 2),
        'Variance': round(s.var(), 2),
        'Minimum': round(s.min(), 2),
        'Année_Min': int(yr_min),
        'Maximum': round(s.max(), 2),
        'Année_Max': int(yr_max),
        'Évolution_globale_%': round(evol, 2),
        'TCAM_%': round(tcam, 2)
    })
df_stats = pd.DataFrame(stat_rows)

# ==============================================================================
# SECTION 4 : MODÉLISATION ET PRÉVISIONS DÉMOGRAPHIQUES (2020-2030)
# ==============================================================================
print("[4/5] Modélisation prévisionnelle (2020-2030)...")

years_future = np.arange(2020, 2031)
prev_dict = {'Année': years_future}

for col in ['Naissances', 'Décès', 'Taux_natalité', 'Taux_mortalité', 'Ex_Ens', 'Population_milliers']:
    # Modélisation sur les données récentes du 21e siècle (2000-2019)
    sub = df_clean[df_clean['Année'] >= 2000][[col]].dropna()
    y_vals = sub[col].values
    x_vals = np.arange(len(y_vals))

    slope, intercept, _, _, _ = stats.linregress(x_vals, y_vals)
    x_future = np.arange(len(y_vals), len(y_vals) + len(years_future))
    prev_dict[col] = np.round(intercept + slope * x_future, 1)

df_predictions = pd.DataFrame(prev_dict)

# ==============================================================================
# SECTION 5 : RAPPORT DE NETTOYAGE ET ANOMALIES
# ==============================================================================
anom_list = []
check_cols = ['Naissances', 'Décès', 'Taux_natalité', 'Taux_mortalité', 'Accroissement_naturel', 'Population_milliers']
for col in check_cols:
    for idx_row in range(len(df_merged)):
        v = df_merged[col].iloc[idx_row]
        y = int(df_merged['Année'].iloc[idx_row])
        if pd.isna(v):
            corr = df_clean[col].iloc[idx_row]
            anom_list.append({
                'Type': 'Donnée manquante',
                'Colonne': col,
                'Année': y,
                'Valeur_originale': 'VIDE',
                'Valeur_corrigée': round(corr, 1) if not pd.isna(corr) else '',
                'Méthode': 'Interpolation linéaire',
                'Note': 'Donnée absente du PDF ONS'
            })
        elif not pd.isna(v):
            mean_col = df_merged[col].dropna().mean()
            std_col = df_merged[col].dropna().std()
            if std_col > 0 and abs(v - mean_col) / std_col > 2.5:
                anom_list.append({
                    'Type': 'Valeur extrême',
                    'Colonne': col,
                    'Année': y,
                    'Valeur_originale': v,
                    'Valeur_corrigée': 'Maintenue',
                    'Méthode': 'Z-score > 2.5',
                    'Note': 'Valeur historiquement justifiée'
                })
df_anom = pd.DataFrame(anom_list) if anom_list else pd.DataFrame(columns=['Type', 'Colonne', 'Année', 'Valeur_originale', 'Valeur_corrigée', 'Méthode', 'Note'])

# ==============================================================================
# SECTION 6 : GÉNÉRATION DU RAPPORT EXCEL STYLISÉ AVEC GRAPHIQUES ONS
# ==============================================================================
print(f"[5/5] Création du classeur Excel : {chemin_excel}")

wb = xlsxwriter.Workbook(chemin_excel)

# Formats Excel professionnels
hdr_fmt = wb.add_format({
    'font_name': 'Arial', 'font_size': 11, 'bold': True, 'bg_color': '#1F4E79',
    'font_color': 'white', 'border': 1, 'align': 'center', 'valign': 'vcenter', 'text_wrap': True
})
cell_w = wb.add_format({'font_name': 'Arial', 'font_size': 10, 'border': 1, 'bg_color': '#FFFFFF', 'num_format': '#,##0'})
cell_b = wb.add_format({'font_name': 'Arial', 'font_size': 10, 'border': 1, 'bg_color': '#D6E4F0', 'num_format': '#,##0'})
cell_wd = wb.add_format({'font_name': 'Arial', 'font_size': 10, 'border': 1, 'bg_color': '#FFFFFF', 'num_format': '0.00'})
cell_bd = wb.add_format({'font_name': 'Arial', 'font_size': 10, 'border': 1, 'bg_color': '#D6E4F0', 'num_format': '0.00'})

title_fmt = wb.add_format({'font_name': 'Arial', 'font_size': 13, 'bold': True, 'font_color': '#1F4E79'})
note_fmt = wb.add_format({'font_name': 'Arial', 'font_size': 9, 'italic': True, 'font_color': '#666666', 'text_wrap': True})
src_fmt = wb.add_format({'font_name': 'Arial', 'font_size': 9, 'italic': True, 'font_color': '#1F4E79'})

def write_df_to_sheet(ws, data, sr=0, sc=0, decimal_cols=[]):
    """Écrit un DataFrame dans une feuille Excel avec des styles professionnels."""
    columns_list = list(data.columns)
    ws.freeze_panes(sr+1, 0)
    for col_idx, col_name in enumerate(columns_list):
        ws.write(sr, sc + col_idx, col_name, hdr_fmt)
        ws.set_column(sc + col_idx, sc + col_idx, max(len(str(col_name)) + 4, 12))

    for row_idx in range(len(data)):
        for col_idx in range(len(columns_list)):
            val = data.iloc[row_idx, col_idx]
            col_name = columns_list[col_idx]
            is_decimal = col_name in decimal_cols

            if row_idx % 2 == 0:
                fmt = cell_bd if is_decimal else cell_b
            else:
                fmt = cell_wd if is_decimal else cell_w

            if pd.isna(val):
                ws.write(sr + row_idx + 1, sc + col_idx, '', fmt)
            else:
                ws.write(sr + row_idx + 1, sc + col_idx, val, fmt)
    return sr + len(data) + 2

# --- F1 - Données Originales ---
s1 = wb.add_worksheet('F1-Données Originales')
s1.write(0, 0, 'DONNÉES ORIGINALES - Source : ONS, Rétrospective Statistique 1962-2020', title_fmt)
s1.write(1, 0, 'Ces données brutes ne doivent JAMAIS être modifiées.', note_fmt)
s1.write(3, 0, 'Tableau 1 : Mouvement naturel de la population', title_fmt)
r = write_df_to_sheet(s1, df1, sr=4)
s1.write(r, 0, 'Tableau 2 : Indicateurs démographiques', title_fmt)
r = write_df_to_sheet(s1, df2, sr=r+1, decimal_cols=['Taux_natalité', 'Taux_mortalité', 'Accroissement_naturel'])
s1.write(r, 0, 'Tableau 40 : Population résidente', title_fmt)
r = write_df_to_sheet(s1, df40, sr=r+1)
s1.write(r, 0, 'Tableau 41 : Population par sexe', title_fmt)
r = write_df_to_sheet(s1, df41, sr=r+1)
s1.write(r, 0, 'Espérance de vie et mortalité infantile (extraits des tables)', title_fmt)
write_df_to_sheet(s1, df_esp, sr=r+1, decimal_cols=['Ex_Ens', 'Ex_H', 'Ex_F', 'Qx_Ens', 'Qx_H', 'Qx_F'])

# --- F2 - Démographie Générale ---
s2 = wb.add_worksheet('F2-Démographie Générale')
s2.write(0, 0, 'DONNÉES DÉMOGRAPHIQUES GÉNÉRALES (1963-2019)', title_fmt)
s2.write(1, 0, 'Source : ONS - Tableaux 1, 2 et 40 fusionnés', src_fmt)
cols2 = ['Année', 'Population_milliers', 'Naissances', 'Décès', 'Mort_nés', 'Mariages', 'Taux_natalité', 'Taux_mortalité', 'Accroissement_naturel']
write_df_to_sheet(s2, df_clean[cols2], sr=3, decimal_cols=['Taux_natalité', 'Taux_mortalité', 'Accroissement_naturel'])

# --- F3 - Natalité ---
s3 = wb.add_worksheet('F3-Natalité')
s3.write(0, 0, 'ANALYSE DE LA NATALITÉ EN ALGÉRIE (1963-2019)', title_fmt)
df3 = df_clean[['Année', 'Naissances', 'Taux_natalité']].copy()
df3['Évolution_annuelle_%'] = df3['Naissances'].pct_change() * 100
write_df_to_sheet(s3, df3, sr=2, decimal_cols=['Taux_natalité', 'Évolution_annuelle_%'])

ch1 = wb.add_chart({'type': 'line'})
ch1.add_series({
    'categories': ['F3-Natalité', 3, 0, 3+len(df3)-1, 0],
    'values': ['F3-Natalité', 3, 1, 3+len(df3)-1, 1],
    'name': 'Naissances', 'line': {'color': '#2E75B6', 'width': 2.5}
})
ch1.set_title({'name': 'Évolution des Naissances en Algérie (1963-2019)'})
ch1.set_x_axis({'name': 'Année'})
ch1.set_y_axis({'name': 'Nombre de naissances'})
ch1.set_size({'width': 720, 'height': 420})
ch1.set_legend({'position': 'bottom'})
s3.insert_chart('F3', ch1)

ch2 = wb.add_chart({'type': 'line'})
ch2.add_series({
    'categories': ['F3-Natalité', 3, 0, 3+len(df3)-1, 0],
    'values': ['F3-Natalité', 3, 2, 3+len(df3)-1, 2],
    'name': 'Taux de natalité (‰)', 'line': {'color': '#C00000', 'width': 2.5}
})
ch2.set_title({'name': 'Évolution du Taux de Natalité (‰)'})
ch2.set_x_axis({'name': 'Année'})
ch2.set_y_axis({'name': 'Taux (‰)'})
ch2.set_size({'width': 720, 'height': 420})
s3.insert_chart('F25', ch2)

# --- F4 - Mortalité ---
s4 = wb.add_worksheet('F4-Mortalité')
s4.write(0, 0, 'ANALYSE DE LA MORTALITÉ EN ALGÉRIE', title_fmt)
df4 = df_clean[['Année', 'Décès', 'Taux_mortalité', 'Qx_Ens']].copy()
df4.columns = ['Année', 'Décès', 'Taux_mortalité_‰', 'Mortalité_infantile_Qx']
write_df_to_sheet(s4, df4, sr=2, decimal_cols=['Taux_mortalité_‰', 'Mortalité_infantile_Qx'])

ch3 = wb.add_chart({'type': 'line'})
ch3.add_series({
    'categories': ['F4-Mortalité', 3, 0, 3+len(df4)-1, 0],
    'values': ['F4-Mortalité', 3, 1, 3+len(df4)-1, 1],
    'name': 'Décès', 'line': {'color': '#404040', 'width': 2.5}
})
ch3.set_title({'name': 'Évolution des Décès en Algérie'})
ch3.set_size({'width': 720, 'height': 420})
s4.insert_chart('F3', ch3)

ch4 = wb.add_chart({'type': 'line'})
ch4.add_series({
    'categories': ['F4-Mortalité', 3, 0, 3+len(df4)-1, 0],
    'values': ['F4-Mortalité', 3, 3, 3+len(df4)-1, 3],
    'name': 'Mortalité infantile (Qx)', 'line': {'color': '#BF0000', 'width': 2.5}
})
ch4.set_title({'name': 'Évolution de la Mortalité Infantile (Qx à 0 an)'})
ch4.set_size({'width': 720, 'height': 420})
s4.insert_chart('F25', ch4)

# --- F5 - Espérance de Vie ---
s5 = wb.add_worksheet('F5-Espérance de Vie')
s5.write(0, 0, 'ESPÉRANCE DE VIE À LA NAISSANCE EN ALGÉRIE', title_fmt)
df5 = df_clean[['Année', 'Ex_H', 'Ex_F', 'Ex_Ens']].copy()
df5.columns = ['Année', 'Homme', 'Femme', 'Ensemble']
df5['Écart_H_F'] = df5['Femme'] - df5['Homme']
write_df_to_sheet(s5, df5, sr=2, decimal_cols=['Homme', 'Femme', 'Ensemble', 'Écart_H_F'])

ch5 = wb.add_chart({'type': 'line'})
ch5.add_series({
    'categories': ['F5-Espérance de Vie', 3, 0, 3+len(df5)-1, 0],
    'values': ['F5-Espérance de Vie', 3, 1, 3+len(df5)-1, 1],
    'name': 'Homme', 'line': {'color': '#2E75B6', 'width': 2.5}
})
ch5.add_series({
    'categories': ['F5-Espérance de Vie', 3, 0, 3+len(df5)-1, 0],
    'values': ['F5-Espérance de Vie', 3, 2, 3+len(df5)-1, 2],
    'name': 'Femme', 'line': {'color': '#C00000', 'width': 2.5}
})
ch5.add_series({
    'categories': ['F5-Espérance de Vie', 3, 0, 3+len(df5)-1, 0],
    'values': ['F5-Espérance de Vie', 3, 3, 3+len(df5)-1, 3],
    'name': 'Ensemble', 'line': {'color': '#548235', 'width': 2, 'dash_type': 'dash'}
})
ch5.set_title({'name': 'Espérance de Vie à la Naissance - Comparaison H/F'})
ch5.set_y_axis({'name': 'Années'})
ch5.set_size({'width': 720, 'height': 420})
s5.insert_chart('G3', ch5)

# --- F6 - Tables de Mortalité ---
s6 = wb.add_worksheet('F6-Tables de Mortalité')
s6.write(0, 0, 'SYNTHÈSE DES TABLES DE MORTALITÉ ABRÉGÉES', title_fmt)
s6.write(1, 0, 'Quotients de mortalité (Qx) et Espérance de vie (Ex) à la naissance', note_fmt)
write_df_to_sheet(s6, df_esp, sr=3, decimal_cols=['Ex_Ens', 'Ex_H', 'Ex_F', 'Qx_Ens', 'Qx_H', 'Qx_F'])

# --- F7 - Population par Sexe ---
s7 = wb.add_worksheet('F7-Population par Sexe')
s7.write(0, 0, 'POPULATION PAR SEXE (en milliers)', title_fmt)
df7 = df_clean[['Année', 'Pop_Masculin', 'Pop_Féminin', 'Pop_Total']].copy()
df7['Ratio_H_F'] = df7['Pop_Masculin'] / df7['Pop_Féminin']
df7['%_Masculin'] = df7['Pop_Masculin'] / df7['Pop_Total'] * 100
df7['%_Féminin'] = df7['Pop_Féminin'] / df7['Pop_Total'] * 100
write_df_to_sheet(s7, df7, sr=2, decimal_cols=['Ratio_H_F', '%_Masculin', '%_Féminin'])

ch7 = wb.add_chart({'type': 'line'})
ch7.add_series({
    'categories': ['F7-Population par Sexe', 3, 0, 3+len(df7)-1, 0],
    'values': ['F7-Population par Sexe', 3, 1, 3+len(df7)-1, 1],
    'name': 'Masculin', 'line': {'color': '#2E75B6', 'width': 2.5}
})
ch7.add_series({
    'categories': ['F7-Population par Sexe', 3, 0, 3+len(df7)-1, 0],
    'values': ['F7-Population par Sexe', 3, 2, 3+len(df7)-1, 2],
    'name': 'Féminin', 'line': {'color': '#C00000', 'width': 2.5}
})
ch7.set_title({'name': 'Évolution de la Population par Sexe'})
ch7.set_size({'width': 720, 'height': 420})
s7.insert_chart('I3', ch7)

# --- F8 - Analyse Statistique ---
s8 = wb.add_worksheet('F8-Analyse Statistique')
s8.write(0, 0, 'ANALYSE STATISTIQUE DESCRIPTIVE', title_fmt)
write_df_to_sheet(s8, df_stats, sr=2, decimal_cols=list(df_stats.columns[1:]))

# --- F9 - Nettoyage & Anomalies ---
s9 = wb.add_worksheet('F9-Nettoyage Anomalies')
s9.write(0, 0, 'RAPPORT DE NETTOYAGE ET DÉTECTION D\'ANOMALIES', title_fmt)
s9.write(1, 0, 'Méthode : Interpolation linéaire pour les valeurs manquantes, Z-score pour les anomalies', note_fmt)
write_df_to_sheet(s9, df_anom, sr=3)

# --- F10 - Prévisions ---
s10 = wb.add_worksheet('F10-Prévisions 2020-2030')
s10.write(0, 0, 'PROJECTIONS DÉMOGRAPHIQUES 2020-2030', title_fmt)
s10.write(1, 0, 'Méthode : Régression linéaire sur données 2000-2019', note_fmt)
write_df_to_sheet(s10, df_predictions, sr=3, decimal_cols=['Taux_natalité', 'Taux_mortalité', 'Ex_Ens'])

ch10 = wb.add_chart({'type': 'line'})
ch10.add_series({
    'categories': ['F10-Prévisions 2020-2030', 4, 0, 14, 0],
    'values': ['F10-Prévisions 2020-2030', 4, 6, 14, 6],
    'name': 'Population projetée', 'line': {'color': '#2E75B6', 'width': 2.5}
})
ch10.set_title({'name': 'Projection de la Population Algérienne (2020-2030)'})
ch10.set_size({'width': 720, 'height': 420})
s10.insert_chart('I3', ch10)

wb.close()
print(f"=== FICHIER EXCEL GÉNÉRÉ AVEC SUCCÈS ===")
print(f"[-] Chemin du classeur : {chemin_excel}")

# ==============================================================================
# OPTIONNEL : VISUALISATION JUPYTER DIRECTE (Si matplotlib est installé)
# ==============================================================================
try:
    import matplotlib.pyplot as plt
    import seaborn as sns

    # Configuration esthétique pour les graphiques
    sns.set_theme(style="whitegrid")
    plt.rcParams['font.sans-serif'] = 'Arial'
    plt.rcParams['font.family'] = 'sans-serif'

    print("\n[Optionnel - Jupyter] Affichage des courbes clés démographiques...")

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    # Graphique 1 : Naissances
    sns.lineplot(data=df_clean, x='Année', y='Naissances', ax=axes[0,0], color='#2e75b6', linewidth=2.5)
    axes[0,0].set_title("1. Évolution des Naissances (1963-2019)", fontsize=12, fontweight='bold', color='#1f4e79')
    axes[0,0].set_xlabel("Année")
    axes[0,0].set_ylabel("Nombre de naissances")

    # Graphique 2 : Taux Natalité / Mortalité
    sns.lineplot(data=df_clean, x='Année', y='Taux_natalité', ax=axes[0,1], color='#c00000', linewidth=2.5, label='Taux de natalité')
    sns.lineplot(data=df_clean, x='Année', y='Taux_mortalité', ax=axes[0,1], color='#548235', linewidth=2.5, label='Taux de mortalité')
    axes[0,1].set_title("2. Taux de Natalité & Mortalité (‰)", fontsize=12, fontweight='bold', color='#1f4e79')
    axes[0,1].set_xlabel("Année")
    axes[0,1].set_ylabel("Taux pour 1000 habitants")
    axes[0,1].legend()

    # Graphique 3 : Espérance de Vie Homme/Femme
    sns.lineplot(data=df_clean, x='Année', y='Ex_H', ax=axes[1,0], color='#2e75b6', linewidth=2, label='Homme')
    sns.lineplot(data=df_clean, x='Année', y='Ex_F', ax=axes[1,0], color='#c00000', linewidth=2, label='Femme')
    sns.lineplot(data=df_clean, x='Année', y='Ex_Ens', ax=axes[1,0], color='#548235', linewidth=2.5, linestyle='--', label='Ensemble')
    axes[1,0].set_title("3. Espérance de Vie à la naissance (ans)", fontsize=12, fontweight='bold', color='#1f4e79')
    axes[1,0].set_xlabel("Année")
    axes[1,0].set_ylabel("Espérance de vie (ans)")
    axes[1,0].legend()

    # Graphique 4 : Prévision de population (2000-2030)
    historique_pop = df_clean[df_clean['Année'] >= 2000][['Année', 'Population_milliers']].copy()
    historique_pop['Type'] = 'Historique'
    proj_pop = df_predictions[['Année', 'Population_milliers']].copy()
    proj_pop['Type'] = 'Projection'
    total_pop = pd.concat([historique_pop, proj_pop])

    sns.lineplot(data=total_pop, x='Année', y='Population_milliers', hue='Type', palette=['#1f4e79', '#ffc000'], linewidth=2.5, ax=axes[1,1])
    axes[1,1].set_title("4. Projections de la Population (2020-2030)", fontsize=12, fontweight='bold', color='#1f4e79')
    axes[1,1].set_xlabel("Année")
    axes[1,1].set_ylabel("Population (en milliers)")

    plt.tight_layout()
    plt.show()

except ImportError:
    print("\n[-] Note : Pour visualiser les graphiques directement sous Jupyter, veuillez installer matplotlib et seaborn :")
    print("    pip install matplotlib seaborn")
```

```
Requirement already satisfied: xlsxwriter in c:\users\hp\anaconda3\lib\site-packages (3.2.9)
[-] Lecture du fichier texte : c:\Users\hp\Downloads\Hakim333\extracted_text.txt
[1/5] Extraction des données démographiques...
[2/5] Structuration, fusion et nettoyage de la base de données...
[3/5] Calcul de l'analyse descriptive et du TCAM...
[4/5] Modélisation prévisionnelle (2020-2030)...
[5/5] Création du classeur Excel : C:\Users\hp\Memoire_Demographie_Algerie_v3.xlsx
=== FICHIER EXCEL GÉNÉRÉ AVEC SUCCÈS ===
[-] Chemin du classeur : C:\Users\hp\Memoire_Demographie_Algerie_v3.xlsx

[Optionnel - Jupyter] Affichage des courbes clés démographiques...
```



In [4]:

```
pip install XlsxWriter
```

```
Requirement already satisfied: XlsxWriter in c:\users\hp\anaconda3\lib\site-packages (3.2.9)
Note: you may need to restart the kernel to use updated packages.
```